

## SUMMARY

This video explains the principles related to state optimization in Minimal Deterministic Finite Automata (DFA) and the formal logic of complementation.

Core conclusions:

State Count Formulas (AD Rule):

For constraints involving “exactly” or “at most”  $m$  and  $n$  occurrences of characters, minimal states =  $(m+1)(n+1) + 1$ .

For “at least” constraints, minimal states =  $(m+1)(n+1)$ .

DFA Complementation:

A DFA for language  $L$  can be converted into a DFA for its complement  $L'$  by interchanging final and non-final states. Total states remain unchanged.

NFA Limitation:

Complementation by swapping final and non-final states works only for DFAs. For NFAs, conversion to DFA is mandatory before complementation.

Substring Efficiency:

A minimal DFA recognizing a substring of length  $n$  requires  $n+1$  states. The complement (“does not contain substring”) also requires  $n+1$  states.

## STATE OPTIMIZATION IN COMPOUND AUTOMATA

Determining minimal states for character-count constraints follows structured arithmetic patterns.

### 1. “EXACTLY” AND “AT MOST” CONSTRAINTS

When a language requires exactly  $m$  occurrences of ‘a’ and  $n$  occurrences of ‘b’, the DFA must track all valid combinations plus a dead state for overflow.

| Constraint Type     | Formula for Minimal States | Example (2 a’s and 4 b’s) |
|---------------------|----------------------------|---------------------------|
| Exactly $m$ and $n$ | $(m+1)(n+1) + 1$           | $(2+1)(4+1) + 1 = 16$     |
| At Most $m$ and $n$ | $(m+1)(n+1) + 1$           | $(2+1)(4+1) + 1 = 16$     |

Dead State Logic:

If the count exceeds the allowed value (e.g., third 'a' when only two allowed), the DFA transitions to a dead state.

Multiple dead combinations in a product machine are merged into a single dead state in the minimal DFA.

## 2. "AT LEAST" CONSTRAINTS

For at least  $m$  occurrences of 'a' and  $n$  of 'b', exceeding the minimum does not invalidate the string.

Formula:

$$\text{Minimal States} = (m+1)(n+1)$$

Example:

At least 2 a's and 4 b's  
 $(2+1)(4+1) = 15$  states

No dead state is required because extra occurrences remain valid.

## THE CONCEPT OF DFA COMPLEMENTATION

If  $L$  is regular, then its complement  $L^c = \Sigma^* - L$  is also regular.

### 1. PROCEDURE FOR DFA COMPLEMENTATION

Given DFA  $M$  for  $L$ :

1. Convert every final state into non-final.
2. Convert every non-final state into final.
3. Keep transitions and initial state unchanged.

### 2. STRUCTURAL PROPERTIES

State Invariance:

If minimal DFA for  $L$  has  $n$  states, minimal DFA for  $L^c$  also has  $n$  states.

Final State Count:

If DFA has  $n$  total states and  $k$  final states, the complement has  $n$  total states and  $(n - k)$  final states.

Set-Theoretic Relations:

$$L \cup L^c = \Sigma^*$$

$$L \cap L^c = \emptyset$$

## CRITICAL DISTINCTION: DFA VS. NFA COMPLEMENTATION

Complementation rules differ fundamentally between DFA and NFA.

### 1. DFA RULE

Swapping final and non-final states in a DFA always produces the complement language.

### 2. NFA EXCEPTION

Swapping final and non-final states in an NFA does not produce the complement language.

Reason:

An NFA accepts a string if at least one path leads to a final state.

Swapping states does not correctly account for missing computation paths.

### 3. CORRECT NFA COMPLEMENT PROCEDURE

1. Convert the NFA into an equivalent DFA.
2. Swap final and non-final states in the resulting DFA.

## SUBSTRING LOGIC AND COMPLEMENTATION

For any substring of length  $n$ :

Minimal DFA States =  $n + 1$

Example:

Substring "0110" (length 4)  
States required =  $4 + 1 = 5$

Complement Case:

Language "does not contain 0110"  
States required = 5

The structure remains the same; only final states differ.

